Open Source CFD

Tutorial #4: 3D Y-Pipe

Incompressible Steady Flow: 3D Y-Pipe with simpleFOAM in terminal

Robert Castilla
Department of Fluid Mechanics

2025-26
Version 1.0
Learning Objectives:

- Create a complex 3D geometry with sketch and additive pipe with FreeCAD
- Generate high quality triangulated surface
- Mesh a 3D geometry with `snappyHexMesh`
- Setup a case with multiple inlets
- Analyze flow and pressure drop
- Visualize 3D results with `paraview`

This tutorial introduces the fundamental workflow of Computational Fluid Dynamics (CFD) for a 3D geometry. We simulate the flow in a Y-shaped pipe junction. You will create the 3D geometry with FreeCAD, generate a high-quality triangulated surface file. Then, with OpenFOAM and CLI (Command Line Interface in terminal), mesh the geometry, run the steady simulation. Finally, post-process of the results will be made with CLI and `paraview`.

Contents

1 Introduction **3**

2 Generation of the geometry **4**
2.1 The solid 4
2.2 The shell 6
2.3 The mesh 8

3 Setup of the case **8**
3.1 Preparation of the surface file 9
3.2 The background mesh 11
3.3 Generation of the mesh 12
3.3.1 The castellated mesh 13
3.3.2 The snappy mesh 14
3.3.3 The mesh with layers 15
3.4 Boundary conditions and physical parameters 17
3.5 Setting up the solver 20
3.6 Solving the flow and main results 21

4 Exercises **21**
4.1 Modify pressure boundary conditions 21
4.2 Modification of outlet pressure 22

1 Introduction

The Y-pipe consists of:

- Main pipe: 250 mm long, 100 mm diameter
- Secondary branch: 500 mm long branch at 90°, 100 mm diameter
- Inlet branch: 250 mm long, 50 mm diameter

An scheme of the Y-pipe is shown in Figure 1

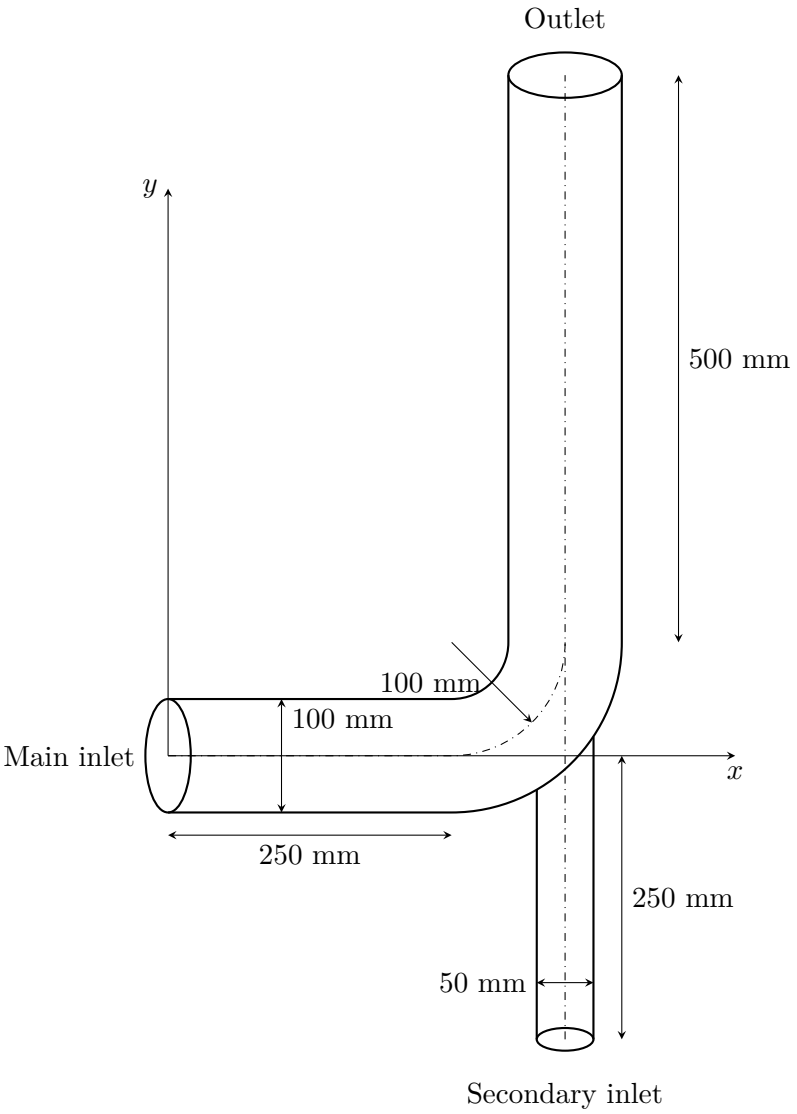

Figure 1: Y-pipe geometry schematic

2 Generation of the geometry

2.1 The solid

First, open a new `Part design` and an sketch in the XY plane. Draw the center line of the main pipe and the branch. It is composed of an horizontal line of 250 mm long, an arc of radius 100 mm and a vertical line 500 mm long. Don't worry about the dimensions and tangent lines when drawing the sketch, they can define later. When the sketch is completely defined, it becomes green, and it can be closed.

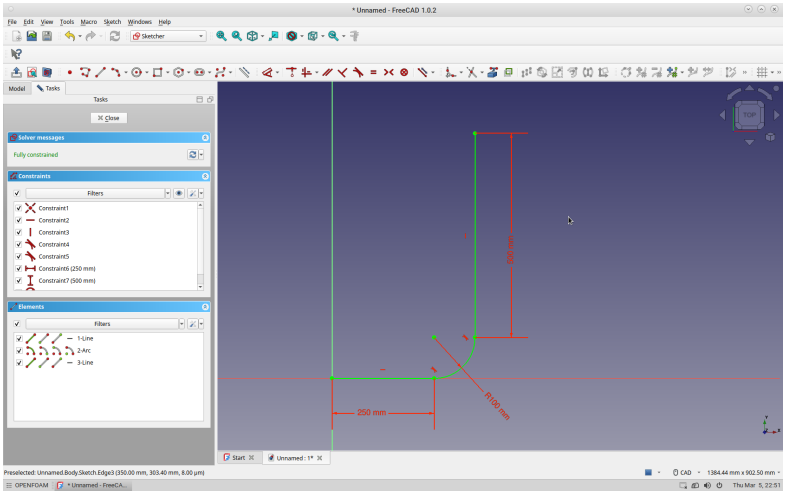

Figure 2: First sketch.

Draw another sketch, with just a circle of radius 50 mm, in the YZ plane

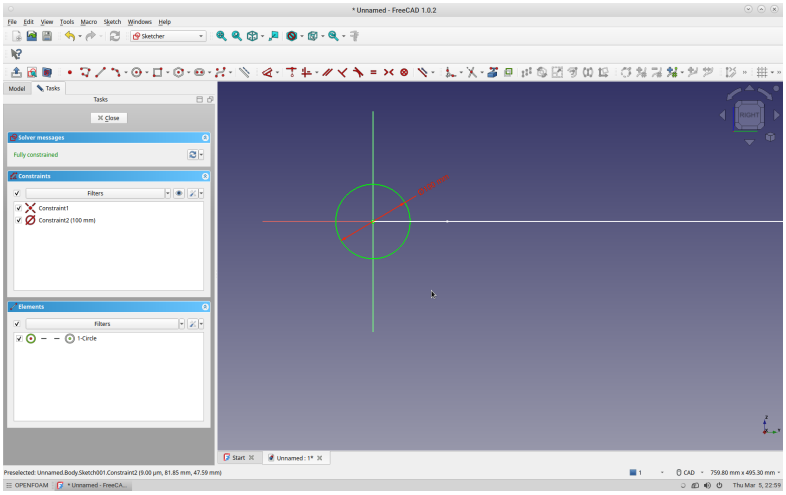

Figure 3: Sketch for 100 mm circle in YZ plane

Select both sketches and make an `Additive pipe`. The result should be like in Figure 4.

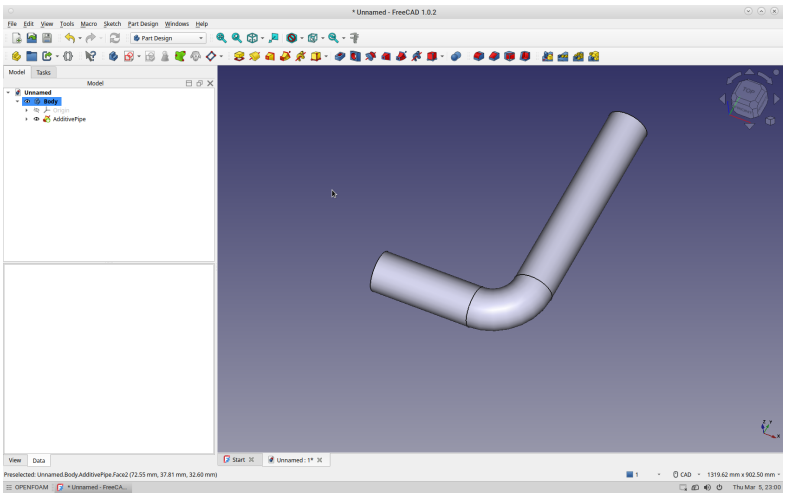

Figure 4: First main pipe

Let's now make the secondary branch. Create another sketch, but in the `outlet` face, with a 50 mm diameter circle, with center coincident to x axis, and 350 mm away from z axis (see Figure 5)

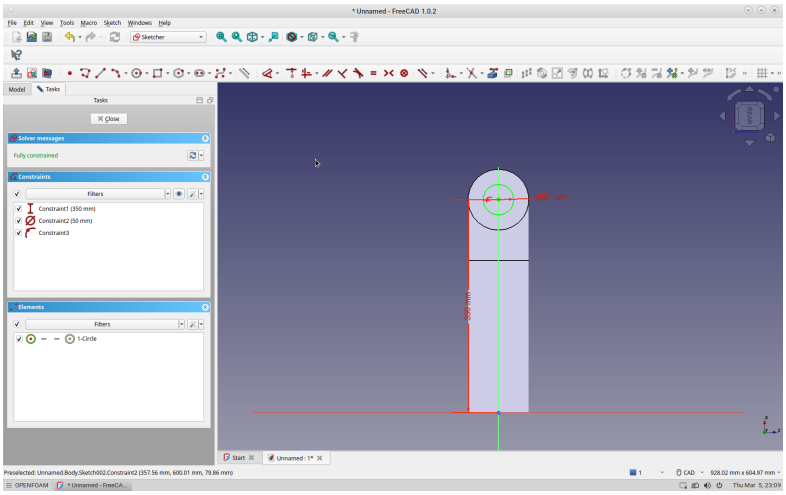

Figure 5: Circle in outlet face.

Make a `Pad` of 800 mm with this circle with reversed direction, and the geometry is finished. **Don't forget to save the work!**. Save it with the name `Ypipe` in a `Tutorial4` folder.

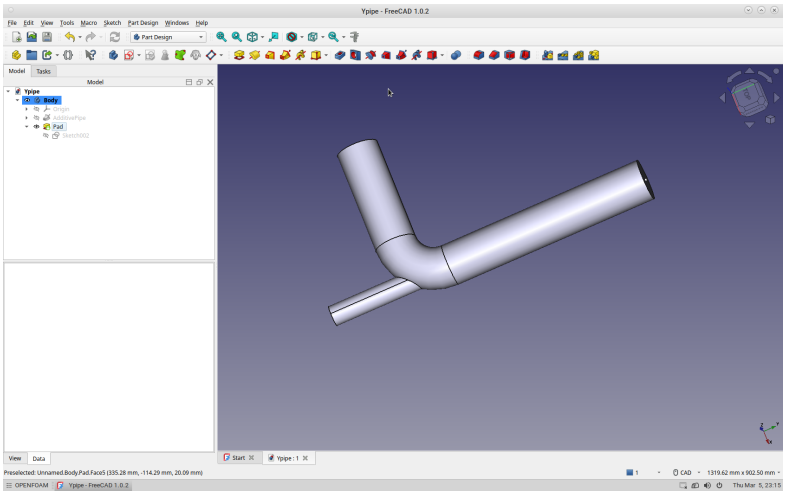

Figure 6: Y-pipe geometry.

2.2 The shell

So far we have generated the solid, but OpenFOAM needs to read the mesh, that is, the surface around the solid, with a proper format. The most usual formats for this surface are Stereolithography (STL) and Wavefront OBJ. The latter is usually the most convenient, specially for large and complex geometries, because the smaller size of the file.

First, explode the `Pad` part with the **`Explode compound`** tool in the `Part` workbench (see Figure 7)

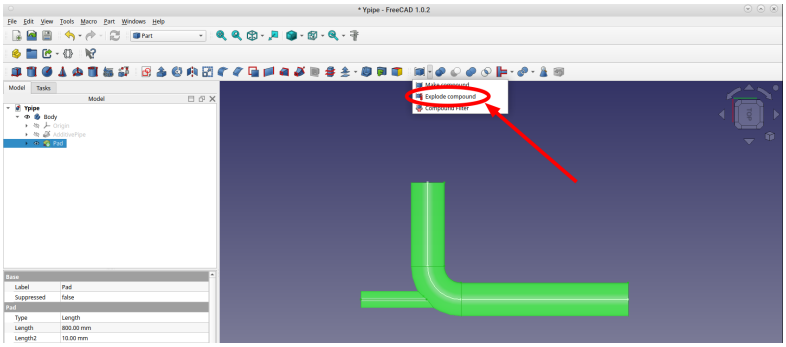

Figure 7: First explode of `Pad` part.

A `Pad.0` element, with the shell of the part, is created. The **`Explode compound`** tool has to be applied on this element again in order to get the seven faces of the solid.

With `F2`, or **`Rename`** option in menu, the names of the faces can be modified:

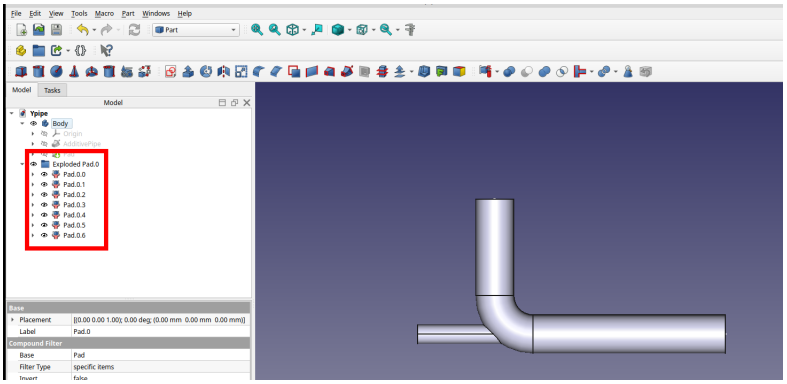

Figure 8: Second explode of the `Pad` part.

- `Pad.0` → `inlet1`
- `Pad.5` → `inlet2`
- `Pad.6` → `outlet`

The four elements `Pad.0.X` that compose the wall shell have to be grouped in an only element with the `Make compound` command

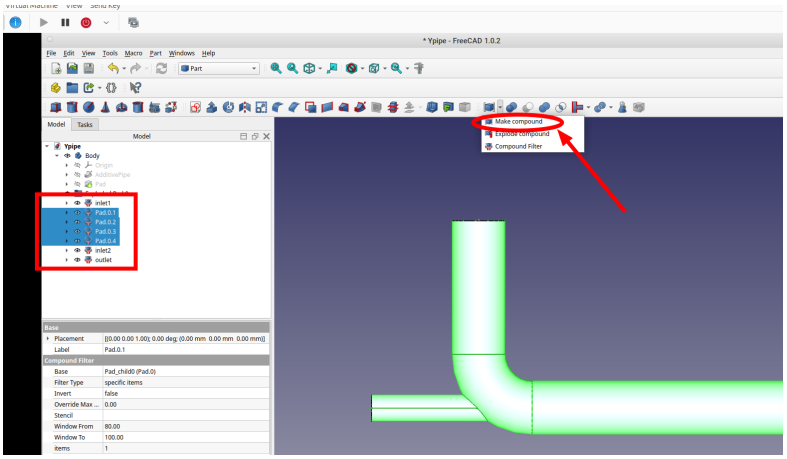

Figure 9: Making compound for the wall patch

Rename this new element

- `Compound` → `wall`

2.3 The mesh

Open the `Shell` workbench and create a mesh from the four shapes. Use `Mefisto` with 10 mm of `Maximum edge length`

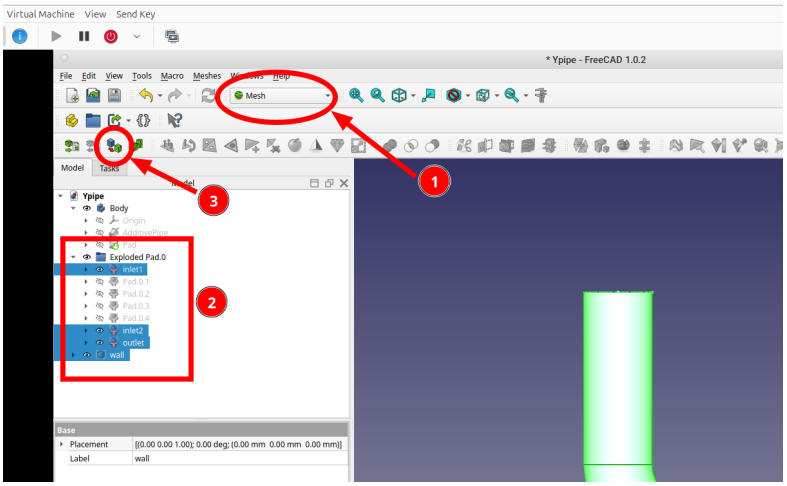

Figure 10: Creating a mesh from the shapes.

This four meshes can now be exported as `ASCII STL`, removing the `(Meshed)` part in the file name. It has to be done with the meshes one by one (see Figure 11). Put the meshes in the folder `/Tutorials/Tutorial_4`, creating it if needed.

The part with `FreeCAD` is over. Now we continue with terminal commands. The session in `FreeCAD` can be saved and the program can be closed.

3 Setup of the case

Like in Tutorial #3, simulation of orifice plate, we are going to use the `inflowOutflow` template case.

```
# load OpenFOAM if it is not included in the .bashrc file
source /usr/lib/openfoam/openfoam-2312/etc/bashrc

# Copy the inflowOutflow in this folder
cp -r $FOAM_ETC/inflowOutflow ~/Tutorials/Tutorial_3/Ypipe
# Note that we have changed the name with the copy

# Go to the tutorial place
cd ~/Tutorials/Tutorial_3/Ypipe
```

We are going to mesh the Ypipe geometry with **`snappyHexMesh`**. Read the `README` file in the case for details.

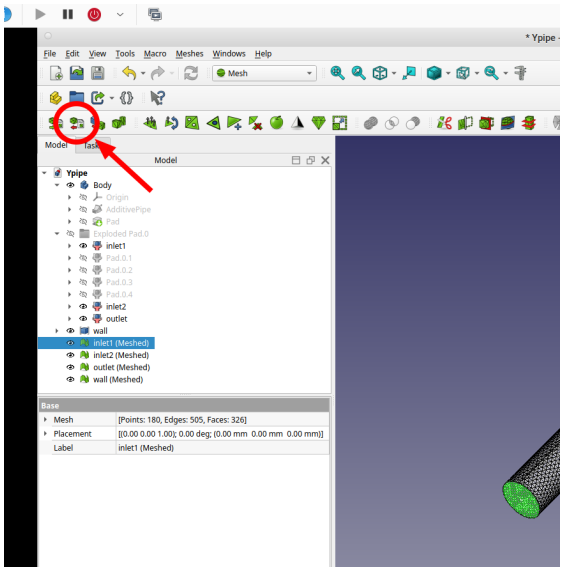

Figure 11: Exporting the meshes

snappyHexMesh is very powerful and the result is usually a high quality mesh. But it requires skills, time and patience in order to use it correctly.

The presentation [1] gives a very detailed description of the program, although a little outdated. Web page [2] is also a useful document with hints for using **snappyHexMesh**

Basically, in order to make the mesh **snappyHexMesh** needs three elements in the case:

- A surface file, in format STL or OBJ, placed in the folder `constant/triSurface`.
- A background structured mesh, usually provided by **blockMesh**.
- A dictionary `snappyHexMesDict`, placed in `system` directory, with all the parameter needed by **snappyHexMesh** to make the mesh.

3.1 Preparation of the surface file

Move the four `*.stl` files generated with FreeCAD to the `constant/triSurface` folder and perform the operations listed in Listing 1.

```
# We are considering that the STL files are in the Tutorial_4 folder
mv ../*.stl constant/triSurface

# Go to the folder to continue the work there and remove the template file
cd constant/triSurface
rm CAD.obj

# Convert STL files to OBJ format. The option -clean, obviously cleans the
```

```
# surface file in the process
surfaceConvert -clean inlet1.stl inlet1.obj
surfaceConvert -clean inlet2.stl inlet2.obj
surfaceConvert -clean outlet.stl outlet.obj
surfaceConvert -clean wall.stl wall.obj

# The STL files can now be removed, if desired
rm *.stl

# The four surfaces are joined in only one surface with four regions
surfaceAdd inlet1.obj inlet2.obj Ypipe.obj
surfaceAdd Ypipe.obj outlet.obj Ypipe.obj
surfaceAdd Ypipe.obj wall.obj Ypipe.obj

# The individual OBJ files can now be reomved as well
rm inlet*.obj outlet.obj wall.obj
```

Listing 1: Preparation of the surface file.

WARNING: The surface has been created in `FreeCAD` in **millimeters**, but OpenFOAM works by default in International System, that is, with **meters**. We have to scale the surface

```
# surfaceTrasnfromPoint -scale <scale factor> <input file> <output file>
# scale factor can be a vector or an scalar
# input and output files can be the same
surfaceTransformPoints -scale 0.001 Ypipe.obj Ypipe.obj
```

Listing 2: Scale of the surface file.

Note that the regions in `Ypipe.obj` fiel have still an annoying `(Meshed)` in the name. We can easily get rid of this with the stream editor **`sed`**.

```
# just for checking, print the lines where ther is a (Meshed) region
grep '(Meshed)' Ypipe.obj

# Remove the expression
sed -i 's/(Meshed)//g' Ypipe.obj
```

Listing 3: Removing the `(Meshed)` in the surface file.

TIP: **`sed`** is a simple but powerful program to process text files. In this command, `-i` option is for 'interactive', the output file is the same as the input file, and the expression `'s/<A>/<B>/g'` substitutes `<A>` by `<B>` in all the occurrences in the file.

The last step is just to check the surface file and get the bounding box to use with **`blockMesh`**.

```
# check and print the bounding box
surfaceCheck -blockMesh yPipe.obj
```

Note the output of this command:

```
...

vertices
(
(0 -0.2 -0.05)
(0.4 -0.2 -0.05)
(0.4 0.6 -0.05)
(0 0.6 -0.05)
(0 -0.2 0.05)
(0.4 -0.2 0.05)
(0.4 0.6 0.05)
(0 0.6 0.05)
);
...
```

3.2 The background mesh

With the output of the last command we modify the `system/blockMeshDict` as in listing 4

```
...
backgroundMesh
{
  xMin    0;
  xMax     0.4;
  yMin    -0.2;
  yMax     0.6;
  zMin    -0.05;
  zMax     0.05;
  xCells  20;
  yCells  40;
  zCells  5;
}
...
```

Listing 4: `blockMeshDict` section of variables definition

Create the block mesh and visualize it along with the surface file to check that everything is right.

```
foamJob -s -log-app blockkMesh
paraFoam
```

It should look like in Figure 12.

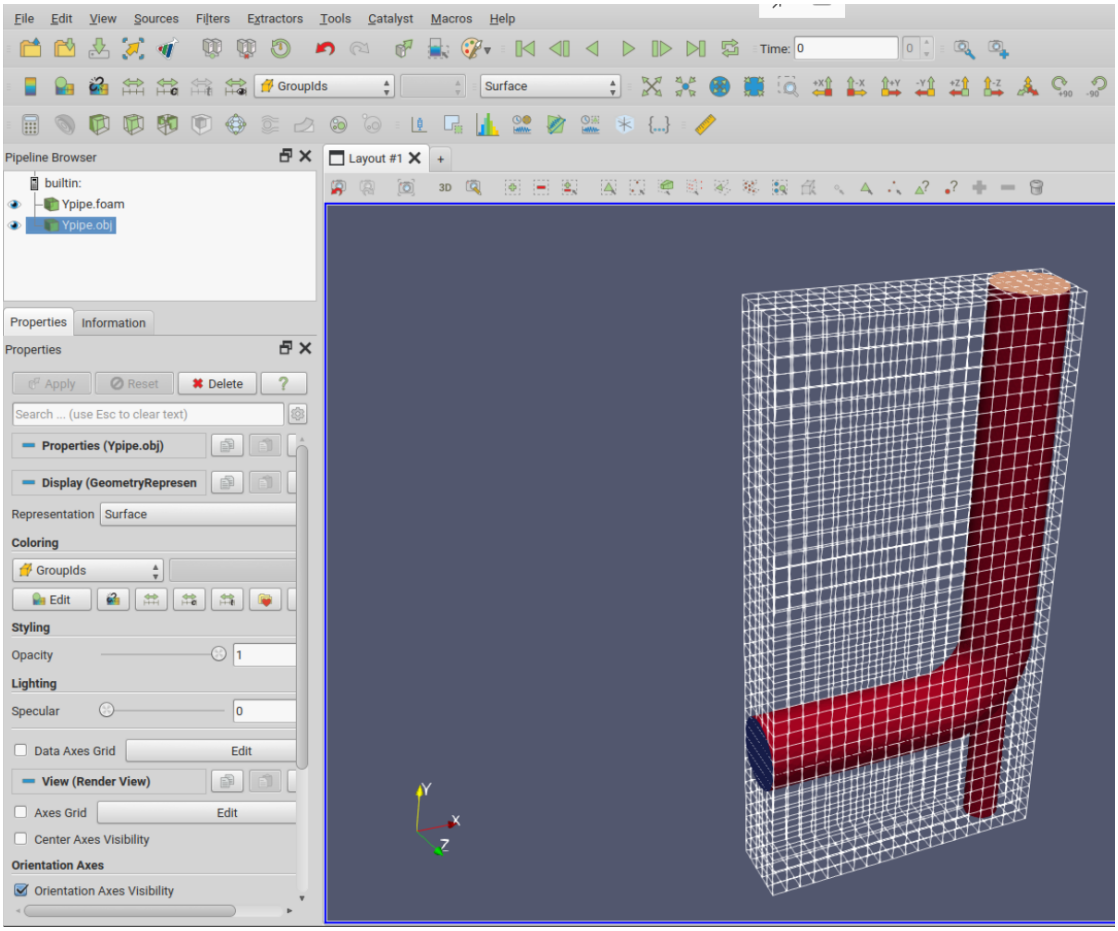

Figure 12: Block mesh and the surface for the geometry

3.3 Generation of the mesh

The program **snappyHexMesh** makes the mesh in three stages:

1. `castellated` It refines the background mesh and defines the patches according to the parameter provided by the dictionary. The result is something reminiscent of Minecraft.
2. `snappy` It deforms the castellated mesh to fit the surface geometry.
3. `addLayer` Insert cell layers to resolve boundary layer in the cells close to the walls.

These stages can be enabled and disabled in the first section of the dictionary

```
    ...
    castellatedMesh  on;
    snap             off;
    addLayers        off;
```

```
    ...
```

3.3.1 The castellated mesh

First, replace all the occurrences of `CAD` with `Ypipe` in the dictionary

```
sed -i 's/CAD/Ypipe/g' system/snappyHexMeshDict
```

Modify the `geometry` and `refinementSurfaces` subdictionaries as shown in Listing 5. the

```
...
geometry
{
  Ypipe.obj
  {
    type triSurfaceMesh;
    name Ypipe;
    regions
    {
      inlet1  { name inlet1; }
      inlet2  { name inlet2; }
      outlet  { name outlet; }
      wall    { name wall;}
    }
  }
};

...

refinementSurfaces
{
  Ypipe
  {
    level (2 2);
    patchInfo { type wall; }

    regions
    {
      "inlet.*"
      {
        level (2 2);
        patchInfo
        {
          type patch;
          inGroups (inlet);
        }
      }
```

```
        outlet
        {
          level (2 2);
          patchInfo
          {
            type patch;
            inGroups (outlet);
          }
        }
      }
    }
  }
```

Listing 5: `geometry` subdictionary

TIP: For the refinement level definition, `(n m)` stands for `n` levels of refinement in the flat regions and `m` levels in corners.

The rest of the `castellatedMesh` subdictionary can be keep as it is.

Run the mesher to perform this first stage.

```
foamJob -s -log-app snappyHexMesh

# move the log to avoid the overwritting by other stages
mv log.snappyHexMesh log.castellated
```

The result is written in folder 1, and can be displayed with **paraFoam** (see Figure 13). Probably you will understand now the reference to Minecraft.

3.3.2 The snappy mesh

In order to perform the second stage, just enable it and disable the first one,

```
    ...
    castellatedMesh  off;
    snap             on;
    addLayers        off;
    ...
```

and run again the mesher

```
    foamJob -s -log-app snappyHexMesh

    # move the log to avoid the overwritting by other stages
    mv log.snappyHexMesh log.snappy
```

Now the result is saved in the folder 2, and it looks like in the Figure 14.

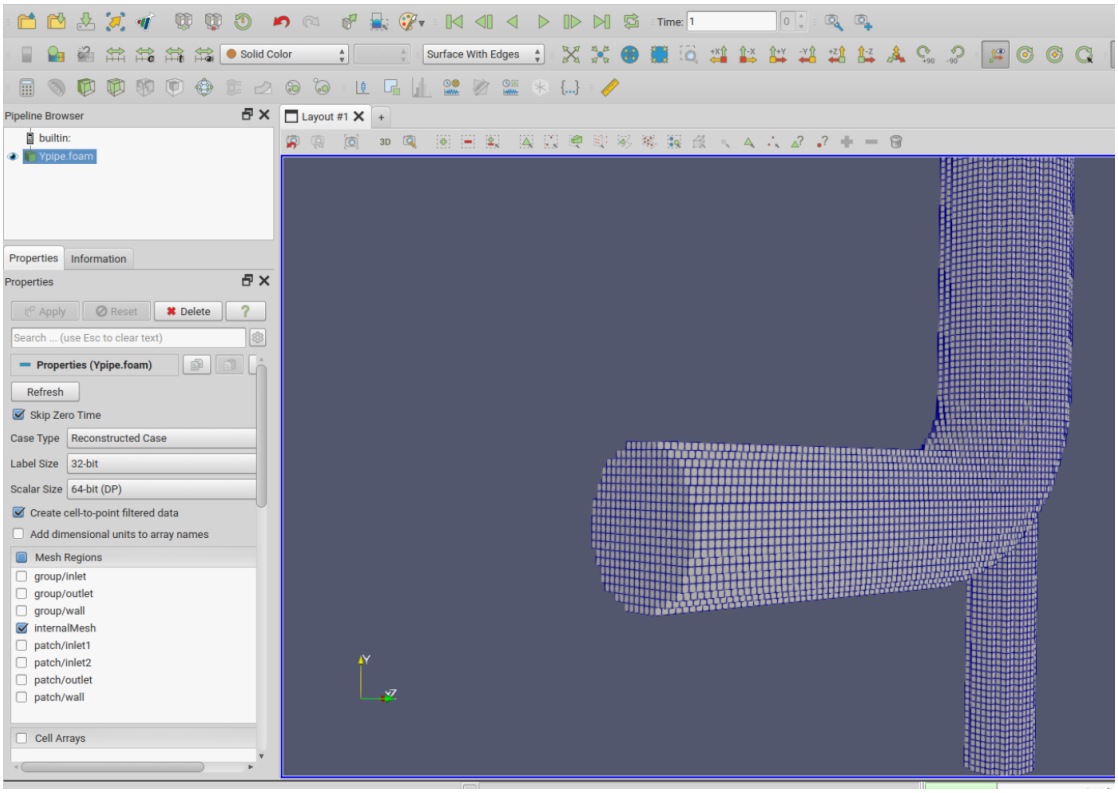

Figure 13: Result of `castellated` stage.

3.3.3 The mesh with layers

The last stage is done by enabling and modifying the `addLayerControls` subdictionary

```
    ...
    castellatedMesh  off;
    snap             off;
    addLayers        on;
    ...
    addLayersControls
    {
      layers
      {
        "wall.*"
        {
          nSurfaceLayers 3;
        }
      }

      relativeSizes       true; // false, usually with
         firstLayerThickness
```

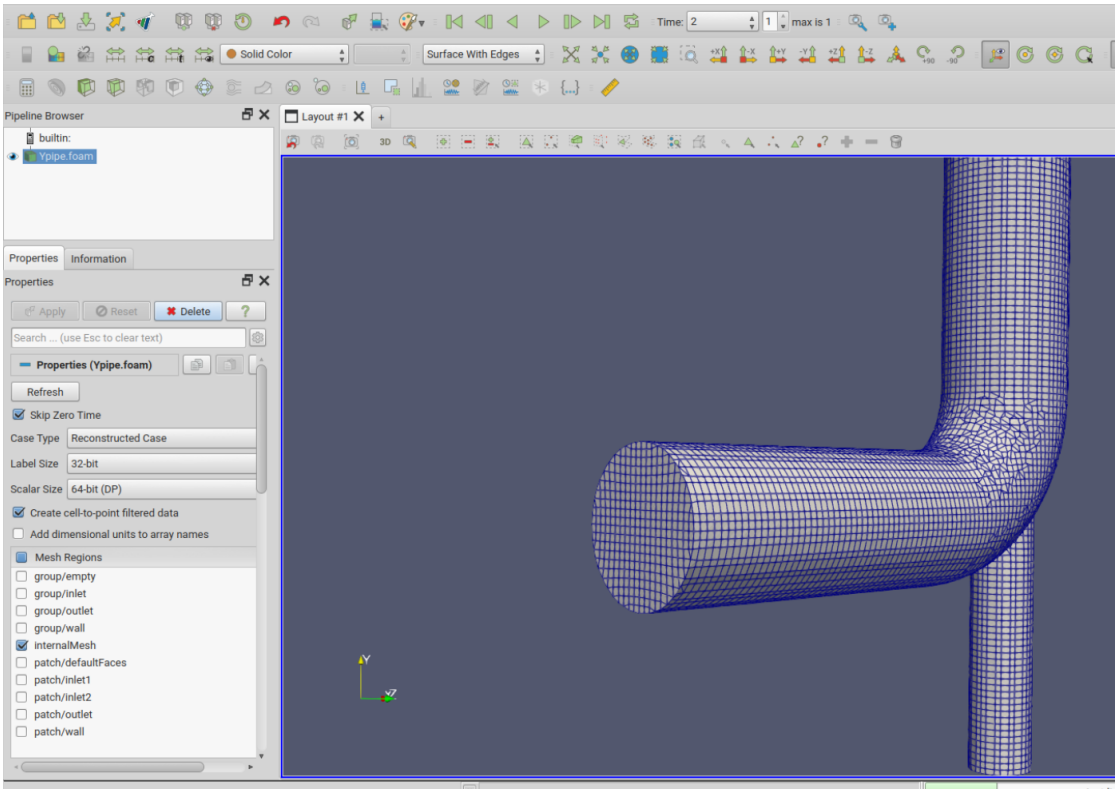

Figure 14: Result of `snappy` stage.

```
    expansionRatio      1.2;
    finalLayerThickness 0.5;
    minThickness        1e-3;
    //  firstLayerThickness 0.01;

    //  maxThicknessToMedialRatio 0.6;
  }
```

```
foamJob -s -log-app snappyHexMesh

# move the log to avoid the overwritting by other stages
mv log.snappyHexMesh log.layers
```

The result is shown in Figure 15.

This mesh can now be placed in the `constant` folder to be used with the simulation, and the intermediate stages can be removed

```
# Remove the block mesh
rm -r constant/polyMesh
```

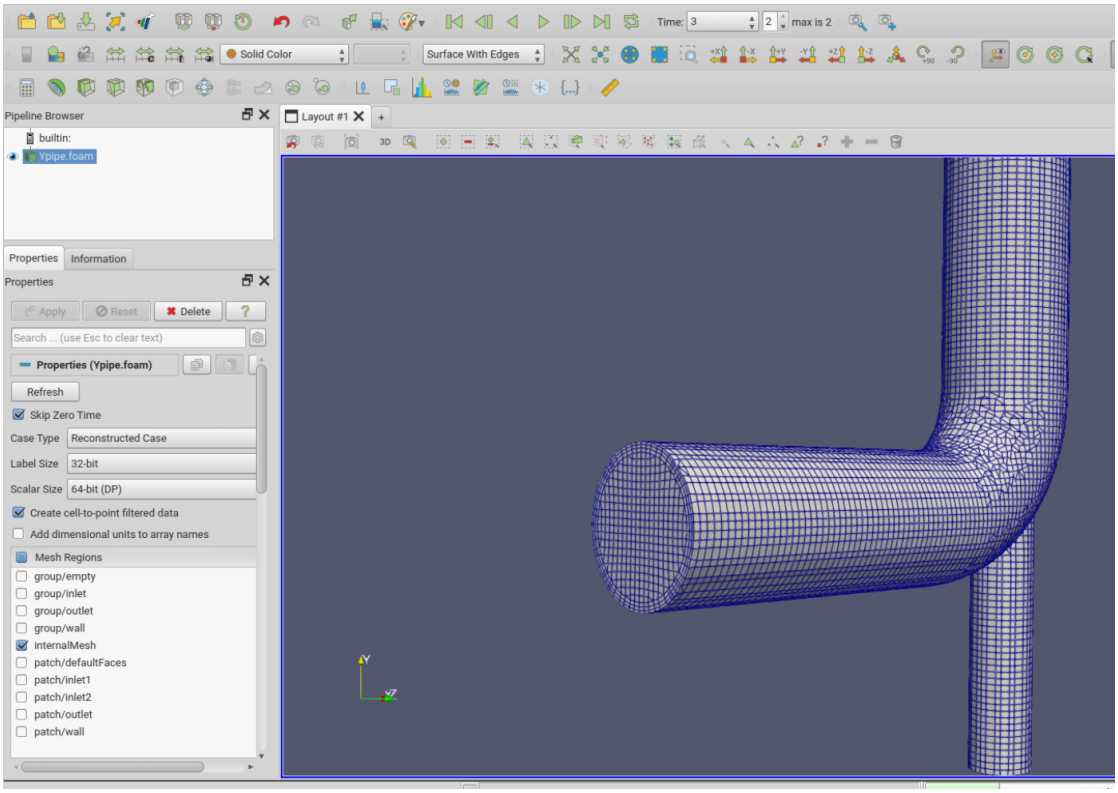

Figure 15: The mesh with layers in the wall.

```
# ... and replaced it by the definitive mesh
mv 3/polyMesh constant
```

TIP: `snappyHexMesh` can be ran with the option `-overwrite` and all the stages `on`. Then it performs the three stages and overwrites the final result on `constant`. We have done this step by step for educational purposes and to shown how to control each stage.

3.4 Boundary conditions and physical parameters

We have to add another `inlet` in all the files of the folder `0`.

```
Uinlet          (0 4 0);

dimensions      [0 1 -1 0 0 0 0];

internalField   uniform (0 0 0);

boundaryField
{
```

```
    inlet1
    {
      type            pressureInletOutletVelocity;
      value           uniform (0 0 0);
    }

    inlet2
    {
      type            fixedValue;
      value           uniform $Uinlet;
    }

    outlet
    {
      type            pressureInletOutletVelocity;
      value           uniform (0 0 0);
    }

    wall
    {
      type            noSlip;
    }

    #includeEtc "caseDicts/setConstraintTypes"
}
```

Listing 6: 0/U

```
dimensions      [0 2 -2 0 0 0 0];

internalField   uniform 0;

boundaryField
{
    inlet1
    {
      type            fixedValue;
      value           uniform 0;
    }

    inlet2
    {
      type            zeroGradient;
    }

    outlet
    {
      type            fixedValue;
      value           uniform 0;
    }
```

```
    wall
    {
      type            zeroGradient;
    }

    #includeEtc "caseDicts/setConstraintTypes"
}
```

Listing 7: 0/p

```
kInlet          0.00375;

dimensions      [0 2 -2 0 0 0 0];

internalField   uniform $kInlet;

boundaryField
{
    inlet1
    {
        type            inletOutlet;
        inletValue      uniform $kInlet;
        value           uniform $kInlet;
    }

    inlet2
    {
        type            turbulentIntensityKineticEnergyInlet;
        intensity       0.05;
        value           uniform $kInlet;
    }

    outlet
    {
        type            inletOutlet;
        inletValue      uniform $kInlet;
        value           uniform $kInlet;
    }

    wall
    {
        type            kqRWallFunction;
        value           uniform $kInlet;
    }

    #includeEtc "caseDicts/setConstraintTypes"
}
```

Listing 8: 0/k

```
omegaInlet      80.0;

dimensions      [0 0 -1 0 0 0 0];

internalField   uniform $omegaInlet;

boundaryField
{
  inlet1
  {
    type            inletOutlet;
    inletValue      uniform $omegaInlet;
    value           uniform $omegaInlet;
  }

  inlet2
  {
    type            turbulentMixingLengthFrequencyInlet;
    mixingLength    0.0035;
    value           uniform $omegaInlet;
  }

  outlet
  {
    type            inletOutlet;
    inletValue      uniform $omegaInlet;
    value           uniform $omegaInlet;
  }

  wall
  {
    type            omegaWallFunction;
    value           uniform $omegaInlet;
  }

  #includeEtc "caseDicts/setConstraintTypes"
}
```

Listing 9: 0/omega

The fluid is water, like in Tutorial #3. Modify `constant/transportProperties` according to that.

3.5 Setting up the solver

Like in Tutorial #3, we include the `solverInfo` function with the variables `k` and `omega` added to the dictionary, and the function called from `system/controlDict`. Also, we

include the `flowRatePatch` function, defining the patch `inlet1` in order to monitor the flow rate dragged by the flow in `inlet2`.

3.6 Solving the flow and main results

Run the solver with

```
foamjob -s -log-app simpleFoam
```

The residuals are not very low, specially for pressure and turbulence magnitudes. This is an usual trend for pressure-driven flows.

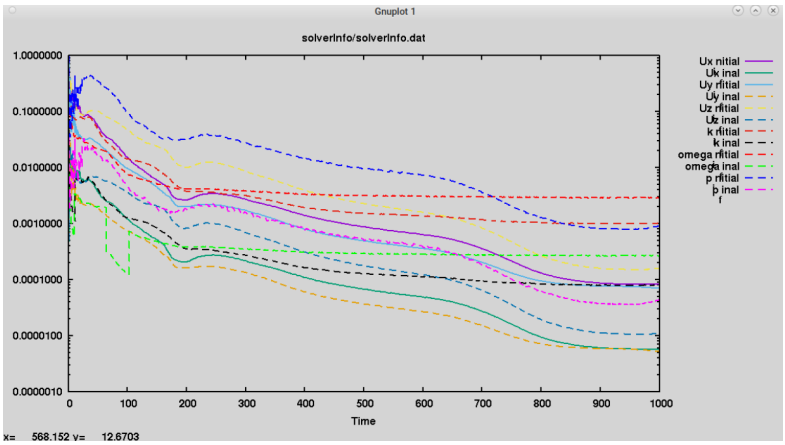

Figure 16: Plot of residuals

The floe rate in `inlet1` converges quite quickly to its final value of, approximately, $-0.008\,\mathrm{m}^3/\mathrm{s}$. The negative value means that it is an inlet flow.

Contours of the four variables are shown in Figures 17 and 18.

4 Exercises

4.1 Modify pressure boundary conditions

The simulation has been conducted with fixed static boundary conditions in both `inlet1` and `outlet`. Nevertheless, it would be more correct to define both boundaries as `totalPressure`. Read the documentation for this boundary condition and run again the simulation with both total pressures with value $0\,\mathrm{m}^2/\mathrm{s}^2$

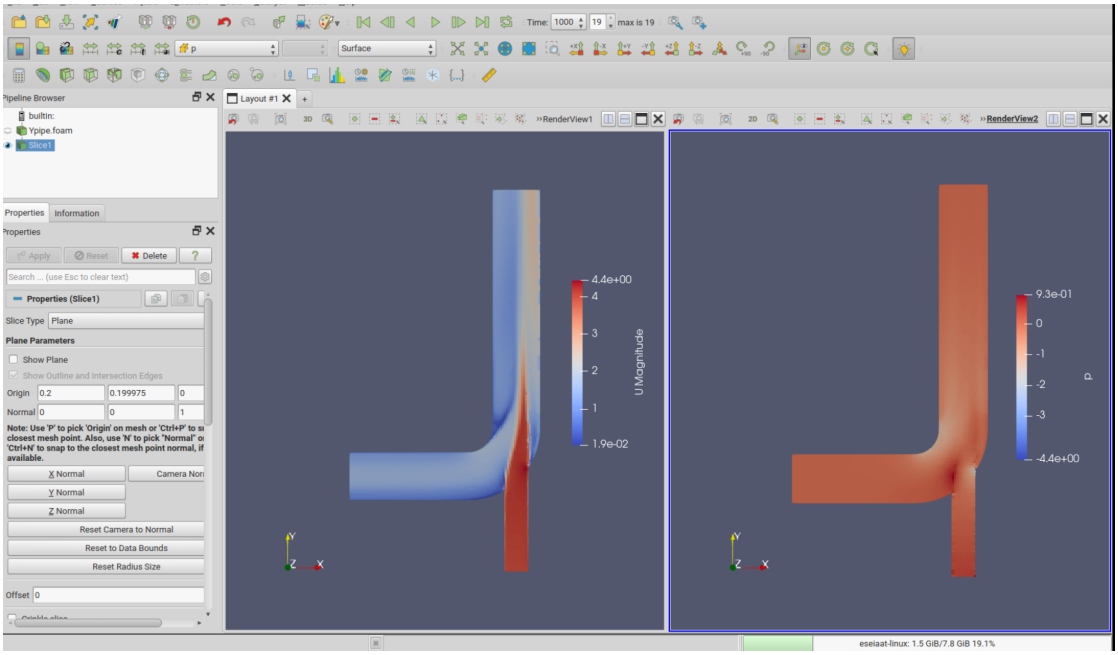

Figure 17: Contours of velocity and pressure in the middle plane.

4.2 Modification of outlet pressure

The outlet pressure has ben defined like the inlet pressure, $0\,\mathrm{m}^2/\mathrm{s}^2$. But this device acts like a pump, where the flow in `inlet1` is dragged against a adverse pressure gradient. Compute the flow rate when the `outlet` total pressure is larger than the inlet total pressure.

Consider that the outlet pressure is $N \times 0.1\,\mathrm{m}^2/\mathrm{s}^2$, where N is the number of group.

References

[1] Jackson A. (2012) *A Comprehensive Tour of snappyHexMesh* 7th OpenFOAM Workshop link (Visited on March 2026)

[2] Thawtar (2024) link (Visited on March 2026)

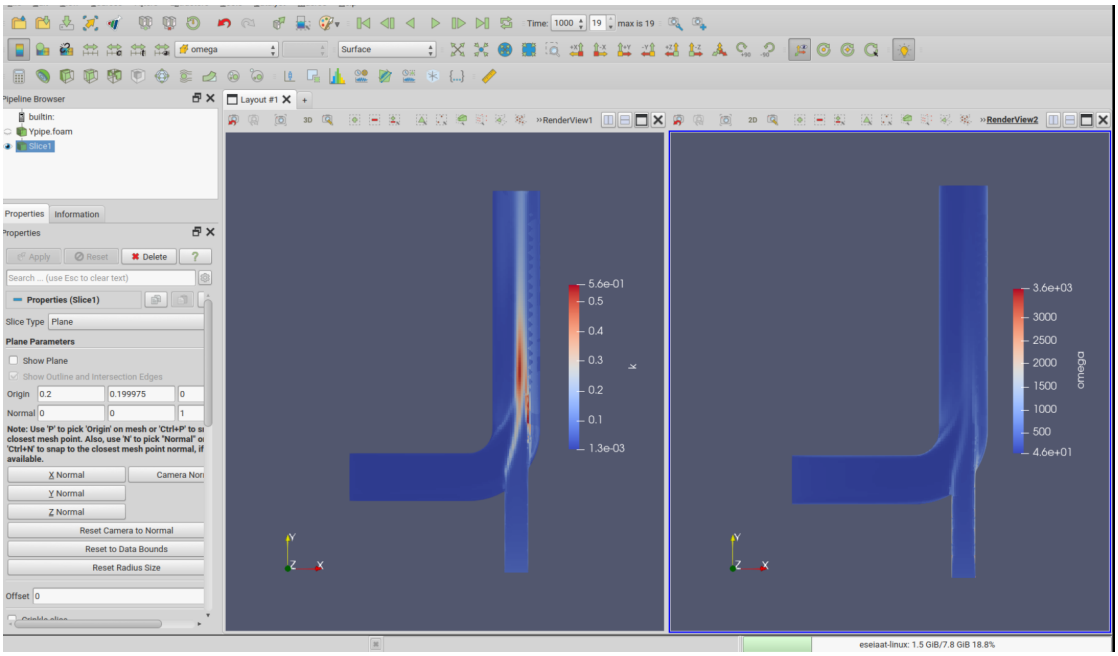

Figure 18: Contours of k and ω in the middle plane.